🚀 ECS Fargate Spot Automatic Failover Solution

CI failing

npm package not found

A fully automated serverless architecture solution for monitoring ECS Fargate Spot instance health and automatically switching to standard Fargate instances during consecutive failures.

🌍 **KEY DIFFERENTIATORS**:

1. **Network Load Balancer (NLB)** - Higher performance and lower latency than ALB
2. **Automatic cross-region disaster recovery** with intelligent health monitoring and DNS-based multi-region failover

License MIT

AWS CDK

TypeScript 4.9+

📋 Table of Contents

- Overview
- Key Features
- Architecture
- Multi-Region Failover
- Quick Start
- Deployment Options
- How It Works
- Monitoring & Alerts
- Cost Analysis
- Troubleshooting
- Cleanup
- Contributing

🎯 Overview

When AWS regions experience failures, Spot resource pools are exhausted, or other reasons cause consecutive Fargate Spot instance startup failures, the system automatically switches workloads to more reliable standard Fargate instances, ensuring high availability of services.

📖 Background Story

This project is the CDK TypeScript implementation of the solution described in the Qiita blog post **「Fargate Spot で月$42K が $21K になった話」** (How we reduced monthly costs from $42K to $21K with Fargate Spot).

- **Blog Post (Japanese)**: Fargate Spot で月$42K が $21K になった話

- **Key Achievement**: Reduced monthly ECS costs by **50%** ($42K → $21K) using Fargate Spot with automatic failover

This CDK implementation upgrades the original Terraform + Python solution to a production-ready TypeScript infrastructure with added NLB (Network Load Balancer) integration, Step Functions orchestration, and enhanced observability.

Use Cases

- Cost-sensitive production environments
- Workloads tolerant of brief interruptions
- Batch processing and background jobs
- Development and testing environments
- Stateless applications requiring auto-recovery

🏭 Application Scenarios

This solution is purpose-built for compute-intensive, long-running workloads where organizations must simultaneously achieve **large-scale cost reduction** and maintain **production-grade availability (≥99% uptime)**. By combining Fargate Spot's cost efficiency with automatic failover to Standard Fargate, the system eliminates the traditional trade-off between cost and reliability.

IoT Data Analytics Pipelines

Large-scale IoT deployments commonly ingest millions of sensor events per hour, requiring persistent, stateful processing containers that run continuously over extended time horizons (hours to weeks). Spot interruptions in this context are particularly disruptive because they can corrupt in-flight aggregation windows and delay time-sensitive anomaly detection.

How this solution addresses it:

- The **Pending Task Monitor** detects capacity exhaustion within 5 minutes — before a single sensor event is dropped — and triggers preemptive failover to Standard Fargate
- **DynamoDB Global Tables** preserve the processing state and error counters across AZs and regions, ensuring no duplicate counting or data loss during the transition
- **Multi-region Route 53 failover** (30-second check interval via `region-health-monitor.ts`) guarantees pipeline continuity even when an entire AWS region experiences a Spot capacity drought
- Typical cost profile: stream processing containers running 24/7 on Fargate Spot at ~70% discount, falling back to Standard only during interruption windows (statistically < 5% of total runtime)

Representative workloads: Industrial sensor aggregation, smart-grid telemetry processing, connected-vehicle data normalization, predictive-maintenance inference pipelines.

Game Backend Compute

Online game backends — physics simulation, match-making, leaderboard calculation, and session state synchronization — impose strict latency and continuity requirements. A Spot reclamation mid-session

results in immediate player-visible disruption, making conventional Spot usage impractical without a robust failover layer.

How this solution addresses it:

- The **Network Load Balancer (Layer 4, ~10–50 µs latency)** preserves active TCP connections during the Spot→Standard switch. Unlike ALB, NLB maintains client IP and connection state, minimizing session disruption to the sub-second window of ECS service stabilization
- The Step Functions **Failover Workflow** (`failover-workflow.asl.json`) polls `RunningCount == DesiredCount && PendingCount == 0` before cutting over, so Standard capacity is fully provisioned before Spot is drained — eliminating any gap in serving capacity
- The **Spot Error Detector** (`spot-error-detector.ts`) distinguishes genuine Spot reclamations (`SpotInterruption`, `ResourcesNotAvailable`, `InsufficientCapacity` patterns) from normal task restarts, preventing unnecessary failovers that would temporarily increase cost
- **Configurable `failureThreshold`** (default: 3 consecutive errors) allows game operators to tune sensitivity based on session tolerance: competitive PvP services can be set to `1` for zero tolerance; background leaderboard jobs can use `5+`

Representative workloads: Authoritative game servers, real-time match-making, physics/AI co-processors, tournament bracket engines, in-game economy settlement.

Business Data Processing & ETL

Enterprise data teams run nightly and continuous ETL jobs — data warehouse ingestion, financial reconciliation, compliance reporting, customer 360 aggregation — that are long-lived but not latency-sensitive. Cost optimization pressure is high, but job failures that require full re-runs are operationally expensive.

How this solution addresses it:

- **Cold Standby mode** for secondary regions keeps cross-region DR cost near zero ($0 for idle Standard tasks) while still providing full recovery capability within 2–3 minutes of a regional incident — appropriate for batch SLAs measured in hours, not seconds
- The **Cleanup Orchestrator** (`cleanup-workflow.asl.json`) executes a staged failback: it waits for Spot stability (`WaitForCleanupDelay`, configurable via `cleanupDelay` input), confirms running count before draining Standard, and updates DynamoDB state atomically — preventing the dual-running cost spike that naive failback approaches incur
- **SNS notifications** at each workflow transition (failover triggered → Standard provisioned → Spot drained → recovery confirmed) give data engineering teams full observability without requiring manual status polling
- **DynamoDB TTL** on error records (30-day default) enables post-incident cost attribution and SLA reporting by preserving the exact timeline of each interruption event

Representative workloads: Nightly data warehouse ETL, financial close batch processing, compliance report generation, ML feature engineering pipelines, CRM data synchronization.

Cost–Availability Reference Matrix

Scenario	Spot Savings	Failover Latency	Recommended Config
IoT streaming (continuous)	~65–70%	45–95 sec	`failureThreshold: 3`, Warm Standby secondary
Game backend (session-critical)	~60–65%	45–60 sec	`failureThreshold: 1`, NLB connection draining
Business ETL (nightly batch)	~70–75%	2–3 min	`failureThreshold: 5`, Cold Standby secondary

> All scenarios maintain **≥99% availability** under typical Spot interruption frequency (AWS SLA: Spot reclamation rate < 5% per instance-hour in standard capacity pools).

🏗 Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                User Requests                                 │
│                                      │                                       │
│                                      ▼                                       │
│                          ┌──────────────────────┐                            │
│                          │   Network Load       │                            │
│                          │     Balancer (NLB)   │                            │
│                          └──────────────────────┘                            │
│                                      │                                       │
│                  ┌───────────────────┴──────────────────────┐                │
│                  │                                          │                │
│                  ▼                                          ▼                │
│   ┌──────────────────┐                      ┌──────────────────┐            │
│   │  Fargate Spot    │     On Failure       │  Fargate         │            │
│   │  (sample-app)    │◄────────────────────►│  (sample-app-    │            │
│   │  • Cost Opt      │     Auto Switch      │   standard)      │            │
│   │  • Replicas: 2   │                      │  • High Rel      │            │
│   └──────────────────┘                      │  • Initial: 0    │            │
│                                             └──────────────────┘            │
│                  │                                                          │
│                  │                                                          │
│                  │  Task State Change Events                                │
│                  ▼                                                          │
│   ┌──────────────────────────────────────────────────────────────────┐      │
│   │                          EventBridge                             │      │
│   │   ┌──────────────┐   ┌──────────────┐   ┌──────────────────────┐ │      │
│   │   │ STOPPED      │   │ RUNNING      │   │ Scheduled (1min)     │ │      │
│   │   │ (Error)      │   │ (Success)    │   │ PENDING Check        │ │      │
│   │   └──────────────┘   └──────────────┘   └──────────────────────┘ │      │
│   └──────────────────────────────────────────────────────────────────┘      │
│                  │                  │                    │                   │
│                  ▼                  ▼                    ▼                   │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐        │
│   │ Spot Error       │   │ Spot Success     │   │ Pending Task     │        │
```

```
│   │ Detector          │   │ Monitor           │   │ Monitor           │              │
│   │ Lambda            │   │ Lambda            │   │ Lambda            │              │
│   │                   │   │                   │   │ (Proactive)       │              │
│   └───────────────────┘   └───────────────────┘   └───────────────────┘              │
│           │                       │                       │                           │
│           ▼                       ▼                       ▼                           │
│   ┌───────────────────┐   ┌───────────────────┐   ┌───────────────────┐             │
│   │ DynamoDB          │   │ Cleanup           │   │ DynamoDB          │             │
│   │ (Counter)         │   │ Orchestrator      │   │ (Counter)         │             │
│   └───────────────────┘   │ Lambda            │   │ (Error++)         │             │
│                           └───────────────────┘   └───────────────────┘             │
│           │                                                                          │
│           ▼                                                                          │
│   ┌───────────────────┐                                                              │
│   │ Failback          │                                                              │
│   │ Orchestrator      │                                                              │
│   │ Lambda            │                                                              │
│   └───────────────────┘                                                              │
│                                                                                      │
│   ┌──────────────────────────────────────────────────────────────────┐               │
│   │ SNS Topic (Notifications)                                         │              │
│   │  • Email  • Slack  • SMS                                          │              │
│   └──────────────────────────────────────────────────────────────────┘               │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

✨ Features

Feature	Description
🚀 **Fully Automated**	Fault detection and switching without manual intervention
🌍 **Multi-Region DR**	**Automatic cross-region failover** with DNS-based routing
💰 **Cost Optimized**	Prioritize Spot instances, save up to 70% on costs
🔄 **Intelligent Recovery**	Auto-switch back to cost-effective mode after Spot recovery
📊 **Real-time Monitoring**	Complete event logs and status tracking
🔔 **Alert Notifications**	Real-time notifications for critical events
⚡ **Fast Response**	Event-driven architecture based on EventBridge
🏗️ **One-Click Deploy**	Complete infrastructure as code with CDK

🌍 Multi-Region Failover (UNIQUE)

This solution provides **true multi-region high availability** - a key differentiator from other ECS Spot failover tools.

Architecture

```
                    Route53 DNS
              (Health Checks + Failover)
                         |
      ┌──────────────────┼──────────────────┐
      │                  │                  │
      ▼                  ▼                  ▼

┌──────────┐       ┌──────────┐       ┌──────────┐
│ Primary  │       │   DR     │       │   DR     │
│us-east-1│       │us-west-2│       │eu-west-1│
├──────────┤       ├──────────┤       ├──────────┤
│ECS Spot  │       │ECS Spot  │       │ECS Spot  │
│   +      │       │   +      │       │   +      │
│Standard  │       │Standard  │       │Standard  │
│          │       │          │       │          │
│DynamoDB  │◄──────►│DynamoDB  │◄──────►│DynamoDB  │
│Global    │       │Global    │       │Global    │
└──────────┘       └──────────┘       └──────────┘
```

Quick Deploy (Multi-Region)

```
# Deploy to 2 regions with automatic DNS failover
npm run deploy -- \
  -c primaryRegion=us-east-1 \
  -c secondaryRegions=us-west-2 \
  -c hostedZoneId=Z123456789 \
  -c dnsRecordName=app.example.com

# Or use warm standby for faster failover
npm run deploy -- \
  -c primaryRegion=us-east-1 \
  -c secondaryRegions=us-west-2,eu-west-1 \
  -c warmStandby=true
```

Key Capabilities

Capability	Description
Health Monitoring	30-second health checks across all regions
Auto Failover	DNS switches to healthy region when primary fails
Auto Failback	Traffic returns to primary when it recovers
State Sync	DynamoDB Global Tables replicate state across regions
Cold/Warm Standby	Choose cost vs. recovery time tradeoff

Comparison

Feature	This Solution	Other Tools
Single Region Spot→Standard	✅	✅
Multi-Region Deployment	✅	❌
Automatic DNS Failover	✅	❌
Cross-Region State Sync	✅	❌
Health-Based Routing	✅	❌

📖 Full Multi-Region Documentation

🚀 Quick Start

Prerequisites

- AWS CLI configured
- Node.js >= 18.x
- AWS CDK >= 2.x
- Required AWS permissions:
  - ECS, Lambda, DynamoDB, EventBridge, SNS, IAM, CloudWatch Logs, ELB

1. Install & Deploy

```
# Clone the repository
git clone https://github.com/weny2000/ecs-spot-failover-cdk.git
cd ecs-spot-failover-cdk

# Install dependencies
npm install

# Deploy full infrastructure (including sample app)
npm run deploy
```

2. Access Sample Application

After deployment, the output will show the load balancer DNS:

```
# Get application URL
aws cloudformation describe-stacks \
  --stack-name EcsFargateSpotFailoverStack \
  --query 'Stacks[0].Outputs[?OutputKey==`LoadBalancerDNS`].OutputValue' \
  --output text
```

Access in browser: `http://<LoadBalancerDNS>`

Sample Application Endpoints

The sample application provides several useful endpoints:

Endpoint	Description
`/`	Application info
`/health`	Health check (used by NLB)
`/status`	Detailed container info
`/simulate-failure`	Trigger test failure

Test the deployment:

```
# Get the Load Balancer DNS
export NLB_DNS=$(aws cloudformation describe-stacks \\
  --stack-name EcsFargateSpotFailoverStack \
  --query 'Stacks[0].Outputs[?OutputKey==`LoadBalancerDNS`].OutputValue' \
  --output text)

# Test endpoints
curl http://$NLB_DNS/
curl http://$NLB_DNS/health
curl http://$NLB_DNS/status

# Simulate failure (container will restart)
curl -X POST http://$NLB_DNS/simulate-failure
```

Using Your Own Application

To deploy your own application instead of the sample nginx app:

```
cd examples/sample-apps/nodejs

# Build and push to ECR
./build.sh v1.0.0 us-east-1 myapp

# Update CDK context to use your image
npm run deploy -- -c appImage=myapp:v1.0.0 -c appPort=8080
```

See examples/sample-apps/nodejs/README.md for detailed instructions.

3. Configure Alerts (Optional)

```
# Add email subscription
aws sns subscribe \
```

```
  --topic-arn $(aws cloudformation describe-stacks \
    --stack-name EcsFargateSpotFailoverStack \
    --query 'Stacks[0].Outputs[?OutputKey==`NotificationTopicArn`].OutputValue'
\
    --output text) \
  --protocol email \
  --notification-endpoint your-email@example.com
```

🔧 Deployment Options

Full Deployment (Recommended)

Deploy complete solution including sample app, load balancer, and monitoring:

```
# Default deployment (2 Spot replicas)
npm run deploy

# Custom replica count
npm run deploy -- -c sampleAppDesiredCount=4

# Custom app port
npm run deploy -- -c appPort=8080
```

Minimal Deployment

Deploy failover mechanism only, without sample app (for existing ECS services):

```
npm run deploy:minimal
```

Step-by-Step Deployment

```
# 1. Compile TypeScript
npm run build

# 2. View changes
npm run diff

# 3. Synthesize CloudFormation template
npm run synth

# 4. Deploy
npm run deploy
```

🔄 How It Works

Normal Operation

```
User Request → NLB → Fargate Spot (sample-app: 2 replicas)
                                ↓
                     Spot Success Monitor (Success Events)
                                ↓
                     Reset Error Counter → Healthy State
```

Failover Flow

```
1. Spot instances fail consecutively (>=3 times)
           ↓
2. EventBridge captures STOPPED events
           ↓
3. Spot Error Detector increments error count
           ↓
4. Threshold reached → Trigger Failback Orchestrator
           ↓
5. Start standard Fargate (sample-app-standard: 2 replicas)
           ↓
6. Stop Spot service (sample-app: 0 replicas)
           ↓
7. NLB automatically switches traffic to standard service
           ↓
8. Send alert notification
```

Auto-Recovery Flow

```
1. Spot instances recover
           ↓
2. Spot Success Monitor detects RUNNING events
           ↓
3. Trigger Cleanup Orchestrator
           ↓
4. Start Spot service (sample-app: 2 replicas)
           ↓
5. Wait for Spot service stabilization
           ↓
6. Stop standard service (sample-app-standard: 0 replicas)
           ↓
7. NLB switches traffic back to Spot service
           ↓
8. Send recovery notification
```

Proactive PENDING Task Monitoring

When Spot capacity is exhausted, tasks may remain in **PENDING** state indefinitely without generating STOPPED events. The **Pending Task Monitor** (runs every 1 minute) proactively detects this:

```
1. Scheduled scan every 1 minute
           ↓
2. List tasks in PENDING state
           ↓
3. Check if tasks stuck for > 5 minutes
           ↓
4. YES → Increment error counter
           ↓
5. If threshold reached → Trigger failover
           ↓
6. NO → Continue normal monitoring
```

This addresses the critical gap in pure event-driven monitoring and ensures rapid failover even when Spot capacity is completely unavailable.

📊 Monitoring & Alerts

CloudWatch Logs

```
# View Spot service logs
aws logs tail /ecs/fargate-spot-sample-app --follow --filter-pattern "spot-service"

# View standard service logs
aws logs tail /ecs/fargate-spot-sample-app --follow --filter-pattern "standard-service"

# View Lambda function logs
aws logs tail /aws/lambda/EcsFargateSpotFailoverStack-SpotErrorDetector --follow
```

DynamoDB Status Query

```
# View error counter
aws dynamodb get-item \
  --table-name fargate-spot-error-counter \
  --key '{"service_name": {"S": "sample-app"}}'
```

Service Status Check

```
# Check Spot service status
aws ecs describe-services \
  --cluster fargate-spot-cluster \
  --services sample-app

# Check standard service status
aws ecs describe-services \
  --cluster fargate-spot-cluster \
  --services sample-app-standard
```

💰 Cost Analysis

Resource Type	Price Estimate	Notes
Fargate Spot	$0.02049/vCPU/hour (us-east-1)	~70% savings vs standard Fargate
Fargate Standard	$0.04048/vCPU/hour	Used during failover
Network Load Balancer	~$0.0225/hour	Lower latency, higher throughput
Lambda	Within free tier	Event-driven, minimal invocations
DynamoDB	Within free tier	On-demand billing
SNS	Within free tier	Alert notifications

Monthly Cost Estimate (2 vCPU config):

- All Spot: ~$18/month
- All Standard: ~$60/month
- Hybrid (90% Spot): ~$22/month

🛠 Troubleshooting

Common Issues

1. Lambda Permission Issues

```
# Check IAM role permissions
aws iam get-role --role-name EcsFargateSpotFailoverStack-LambdaExecutionRole
```

2. EventBridge Rules Not Triggering

```
# Check event rule status
aws events list-rules --name-prefix EcsFargateSpotFailoverStack

# View CloudTrail events
```

```
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=EventName,AttributeValue=PutRule
```

3. Service Creation Failed

```
# Check ECS cluster status
aws ecs describe-clusters --clusters fargate-spot-cluster

# View service events
aws ecs describe-services \
  --cluster fargate-spot-cluster \
  --services sample-app \
  --query 'services[0].events[:5]'
```

4. Failover Not Triggered

Check error counter in DynamoDB:

```
aws dynamodb scan --table-name fargate-spot-error-counter
```

Debug Logs

```
# View all Lambda log groups
aws logs describe-log-groups \
  --log-group-name-prefix "/aws/lambda/EcsFargateSpotFailoverStack"

# Real-time monitoring of all logs
aws logs tail "/aws/lambda/EcsFargateSpotFailoverStack-SpotErrorDetector" --
follow &
aws logs tail "/aws/lambda/EcsFargateSpotFailoverStack-
FargateFailbackOrchestrator" --follow &
aws logs tail "/aws/lambda/EcsFargateSpotFailoverStack-SpotSuccessMonitor" --
follow &
aws logs tail "/aws/lambda/EcsFargateSpotFailoverStack-CleanupOrchestrator" --
follow &
```

🧹 Cleanup

```
# Method 1: Use CDK destroy
npm run destroy

# Method 2: Manually stop services before destroying
aws ecs update-service \
```

```
  --cluster fargate-spot-cluster \
  --service sample-app \
  --desired-count 0

aws ecs update-service \
  --cluster fargate-spot-cluster \
  --service sample-app-standard \
  --desired-count 0

# Wait for services to stop, then
cdk destroy
```

📚 Documentation

Document	Description
Architecture Overview	System architecture and component details
Network Load Balancer	🚀 **High Performance**: NLB configuration and benefits
Deployment Guide	Detailed deployment steps and configuration
Multi-Region Failover	🌍 **Unique feature**: Automatic cross-region disaster recovery
Execution Guide	System execution and operation procedures
Operations Manual	Daily operations, monitoring, and maintenance
Release Guide	Release management and deployment strategies
Testing Guide	Testing procedures and validation strategies
Sample Applications	Ready-to-use sample apps (Node.js, Python, Go) with Docker configurations

🔧 Using as a CDK Construct Library

This project is published as a reusable CDK Construct on npm. You can integrate it into your existing CDK projects:

```
npm install ecs-fargate-spot-failover-cdk
```

Basic Usage

```
import { FargateSpotFailoverConstruct } from 'ecs-fargate-spot-failover-cdk';

// Assuming you have existing ECS services
new FargateSpotFailoverConstruct(this, 'Failover', {
  cluster: myCluster,
  spotService: mySpotService,
```

```
  standardService: myStandardService,
  failureThreshold: 3,
  enableNotifications: true,
  notificationEmails: ['alerts@example.com'],
});
```

Advanced Configuration

```
new FargateSpotFailoverConstruct(this, 'Failover', {
  cluster: myCluster,
  spotService: mySpotService,
  standardService: myStandardService,
  failureThreshold: 5,
  enableNotifications: true,
  notificationEmails: ['ops@example.com'],
  enablePendingTaskMonitoring: true,  // Detect tasks stuck in PENDING
  pendingTaskCheckInterval: Duration.minutes(1),
  pendingTaskTimeout: Duration.minutes(5),
  enableTracing: true,  // X-Ray tracing
  cloudWatchNamespace: 'MyApp/FargateSpot',
});
```

See Construct Hub for full API documentation.

🤝 Contributing

Issues and Pull Requests are welcome!

Development Workflow

```
# 1. Fork and clone
git clone https://github.com/weny2000/ecs-spot-failover-cdk.git

# 2. Create feature branch
git checkout -b feature/your-feature

# 3. Install deps and build
npm install
npm run build

# 4. Run tests
npm test

# 5. Commit changes
git commit -m "feat: add your feature"

# 6. Push and create PR
git push origin feature/your-feature
```

License

This project is open-sourced under the MIT License.

Acknowledgments

- AWS CDK - Infrastructure as Code framework
- AWS Fargate - Serverless container compute

Disclaimer: This project is for learning and reference purposes only. Please test thoroughly before deploying to production.